In [1]:
```python
import re
import glob
import zipfile
import time
from pathlib import Path
import json
import requests
import pandas as pd
from urllib.request import urlretrieve
```

In [2]:
```python
# Measurement helper
import time, psutil, os, functools

_proc = psutil.Process(os.getpid())

def measure(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        m0 = _proc.memory_info().rss / 1e6
        c0 = time.process_time()
        t0 = time.perf_counter()
        out = fn(*args, **kwargs)
        print(f"{fn.__name__}:  wall={time.perf_counter()-t0:.1f}s  "
              f"cpu={time.process_time()-c0:.1f}s  "
              f"mem Δ{_proc.memory_info().rss/1e6 - m0:+.0f} MB")
        return out
    return wrapper
```

1. Downloading the Data

In [3]:
```python
repo_root_dir = Path.cwd().parent
output_dir = repo_root_dir / "data" / "rainfall_over_nsw"
output_dir.mkdir(parents=True, exist_ok=True)
FORCE_DOWNLOAD = False
```

In [4]:
```python
# Necessary metadata
# Dataset used: https://figshare.com/articles/dataset/Daily_rainfall_over_NS
article_id = 14096681  # this is the unique identifier of the article on fig
url = f"https://api.figshare.com/v2/articles/{article_id}"
headers = {"Content-Type": "application/json"}
```

In [ ]:
```python
response = requests.request("GET", url, headers=headers)
response.raise_for_status() # check if the request was successful
data = json.loads(response.text)
files = data["files"]
files
```

Out[ ]:

```
[{'id': 26579150,
  'name': 'daily_rainfall_2014.png',
  'size': 58863,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26579150',
  'supplied_md5': 'fd32a2ffde300a31f8d63b1825d47e5e',
  'computed_md5': 'fd32a2ffde300a31f8d63b1825d47e5e',
  'mimetype': 'image/png'},
 {'id': 26579171,
  'name': 'environment.yml',
  'size': 192,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26579171',
  'supplied_md5': '060b2020017eed93a1ee7dd8c65b2f34',
  'computed_md5': '060b2020017eed93a1ee7dd8c65b2f34',
  'mimetype': 'text/plain'},
 {'id': 26586554,
  'name': 'README.md',
  'size': 5422,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26586554',
  'supplied_md5': '61858c6cc0e6a6d6663a7e4c75bbd88c',
  'computed_md5': '61858c6cc0e6a6d6663a7e4c75bbd88c',
  'mimetype': 'text/x-python'},
 {'id': 26766812,
  'name': 'data.zip',
  'size': 814041183,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26766812',
  'supplied_md5': 'b517383f76e77bd03755a63a8ff83ee9',
  'computed_md5': 'b517383f76e77bd03755a63a8ff83ee9',
  'mimetype': 'application/zip'},
 {'id': 26766815,
  'name': 'get_data.py',
  'size': 4113,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26766815',
  'supplied_md5': '7829028495fd9dec9680ea013474afa6',
  'computed_md5': '7829028495fd9dec9680ea013474afa6',
  'mimetype': 'text/x-python'}]
```


In [ ]:
```python
files_to_dl = ["data.zip"]
for file in files:
    if file["name"] in files_to_dl:

        output_path = output_dir / file["name"]

        if output_path.exists() and not FORCE_DOWNLOAD:
            print(f"{file['name']} already exists. Skipping.")
        else:
            output_path.parent.mkdir(exist_ok=True)

            print(f"Downloading {file['name']}...")
            urlretrieve(file["download_url"], output_path)
```

```
Downloading data.zip...
```

In [8]:
```
zip_path = output_dir / "data.zip"
extract_dir = output_dir / "data"

if not extract_dir.exists():
    print("Extracting files...")
    with zipfile.ZipFile(zip_path, "r") as f:
        f.extractall(output_dir)
else:
    print("Files already extracted. Skipping extraction.")
```

```
Extracting files...
```

2. Combining data CSVs

- exclude file `observed_daily_rainfall_SYD.csv` for this current part of the project
- Create new column based on model name, taken from the file name
  - EX: `SAM0-UNICON_daily_rainfall_NSW.csv` will have a value "SAM0-UNICON" under the new `model` column
- Existing columns: time,lat_min,lat_max,lon_min,lon_max,rain (mm/day)

In [ ]:
```
use_cols = ["time", "lat_min", "lat_max", "lon_min","lon_max","rain (mm/day)
df = pd.read_csv("../data/rainfall_over_nsw/SAM0-UNICON_daily_rainfall_NSW.c
df
```

Out[ ]:

	time	lat_min	lat_max	lon_min	lon_max	rain (mm/day)
0	1889-01-01 12:00:00	-35.811518	-34.86911	140.625	141.875	3.045650e-13
1	1889-01-02 12:00:00	-35.811518	-34.86911	140.625	141.875	3.572392e-04
2	1889-01-03 12:00:00	-35.811518	-34.86911	140.625	141.875	9.431562e-03
3	1889-01-04 12:00:00	-35.811518	-34.86911	140.625	141.875	9.663865e-02
4	1889-01-05 12:00:00	-35.811518	-34.86911	140.625	141.875	0.000000e+00
...	...	...	...	...	...	...
3541148	2014-12-27 12:00:00	-30.157068	-29.21466	153.125	154.375	6.689683e+00
3541149	2014-12-28 12:00:00	-30.157068	-29.21466	153.125	154.375	7.862555e+00
3541150	2014-12-29 12:00:00	-30.157068	-29.21466	153.125	154.375	1.000503e+01
3541151	2014-12-30 12:00:00	-30.157068	-29.21466	153.125	154.375	8.541592e+00
3541152	2014-12-31 12:00:00	-30.157068	-29.21466	153.125	154.375	6.811749e+01

3541153 rows × 6 columns

In [ ]:

```
# Task 2 combine CSVs + add `model`, measured with @measure
combined_dir = repo_root_dir / "data" / "combined"
combined_dir.mkdir(parents=True, exist_ok=True)
output_path = combined_dir / "combined_data.csv"
FORCE_MERGE = False

@measure
def combine_rainfall_csvs(output_path: Path, force_merge: bool = False) -> p
    source_dirs = [Path(output_dir), Path(output_dir) / "data"]
    csv_paths: list[Path] = []
    for d in source_dirs:
        if d.exists():
            csv_paths.extend(sorted(d.glob("*.csv")))

    csv_paths = [
        p for p in csv_paths
        if p.name not in {"observed_daily_rainfall_SYD.csv", output_path.nam
    ]

    if not csv_paths:
        raise FileNotFoundError(
            f"No CSVs found in {[str(d) for d in source_dirs]} (after excluc
```

```
        )

    if output_path.exists() and not force_merge:
        print(f"Merged file already exists at {output_path.name}. Skipping."
        return pd.read_csv(output_path)

    dfs: list[pd.DataFrame] = []
    for p in csv_paths:
        # Model name is the part of the filename before the first underscore
        m = re.search(r"([^_]*)", p.name)
        model = m.group(1) if m else ""

        dfi = pd.read_csv(p, usecols=use_cols)
        dfi.insert(len(dfi.columns), "model", model)
        dfs.append(dfi)

    combined = pd.concat(dfs, ignore_index=True)
    combined.to_csv(output_path, index=False)
    print(f"Wrote {len(combined):,} rows to {output_path.name}")
    return combined

combined = combine_rainfall_csvs(output_path=output_path, force_merge=FORCE_
combined.head()
```

```
Wrote 62,467,843 rows to combined_data.csv
combine_rainfall_csvs:  wall=270.0s  cpu=151.7s  mem Δ-840 MB
```

Out[ ]:

	time	lat_min	lat_max	lon_min	lon_max	rain (mm/day)	model
0	1889-01-01 12:00:00	-36.25	-35.0	140.625	142.5	3.293256e-13	ACCESS-CM2
1	1889-01-02 12:00:00	-36.25	-35.0	140.625	142.5	0.000000e+00	ACCESS-CM2
2	1889-01-03 12:00:00	-36.25	-35.0	140.625	142.5	0.000000e+00	ACCESS-CM2
3	1889-01-04 12:00:00	-36.25	-35.0	140.625	142.5	0.000000e+00	ACCESS-CM2
4	1889-01-05 12:00:00	-36.25	-35.0	140.625	142.5	1.047658e-02	ACCESS-CM2

Task 2 (step 4): compare + reflect

ME(Macbook Air): combine_rainfall_csvs: wall=270.0s, cpu=151.7s, mem Δ=-840 MB
Partner 1 (Windows 11, Snapdragon X1P42100, 16 GB RAM): wall=359.4s, cpu=347.4s, mem Δ=-4 MB
Partner 2 (MacBook Pro): wall=152.9s, cpu=152.5s, mem Δ=+6412 MB

The combine step differs across machines: the MacBook Pro is fastest, my run is in the middle, and Partner 1 is slowest. Memory usage also varies significantly, with one run showing a very large positive increase, suggesting the entire dataset was loaded into memory at once. In my case, the wall time is much higher than CPU time, indicating that

disk I/O is the main bottleneck rather than computation. In contrast, Partner 1 has similar wall and CPU times, suggesting a more CPU-bound process. This highlights how hardware differences, especially disk speed and memory handling, can significantly impact performance when working with large CSV files.

3. Load the Combined CSV to memory and perform EDA

In [ ]:
```
# Task 3 setup: compare memory-aware EDA approaches on combined_data.csv
combined_dir = repo_root_dir / "data" / "combined"
csv_path = combined_dir / "combined_data.csv"
if not csv_path.exists():
    raise FileNotFoundError("combined_data.csv not found in data/combined. R

selected_cols = ["time", "rain (mm/day)", "model"]
dtype_map = {
    "rain (mm/day)": "float32",
    "model": "category",
}

@measure
def eda_dtype_optimized(path: Path):
    """Approach 1: load selected columns with optimized dtypes."""
    df = pd.read_csv(
        path,
        usecols=selected_cols,
        dtype=dtype_map,
        parse_dates=["time"],
    )

    # Simple EDA outputs for comparison
    model_counts = df["model"].value_counts().sort_values(ascending=False)
    rain_summary = df["rain (mm/day)"].describe()
    return model_counts, rain_summary

@measure
def eda_chunked(path: Path, chunk_size: int = 1_000_000):
    """Approach 2: process file in chunks to reduce peak memory."""
    model_counts = pd.Series(dtype="int64")
    rain_count = 0
    rain_sum = 0.0
    rain_sq_sum = 0.0
    rain_min = float("inf")
    rain_max = float("-inf")

    for chunk in pd.read_csv(
        path,
        usecols=selected_cols,
        dtype=dtype_map,
        parse_dates=["time"],
        chunksize=chunk_size,
    ):
```

```
        model_counts = model_counts.add(chunk["model"].value_counts(), fill_

        rain = chunk["rain (mm/day)"].astype("float64")
        rain_count += rain.size
        rain_sum += rain.sum()
        rain_sq_sum += (rain**2).sum()
        rain_min = min(rain_min, rain.min())
        rain_max = max(rain_max, rain.max())

    model_counts = model_counts.sort_values(ascending=False).astype("int64")
    rain_mean = rain_sum / rain_count
    rain_std = ((rain_sq_sum / rain_count) - (rain_mean**2)) ** 0.5

    rain_summary = pd.Series(
        {
            "count": rain_count,
            "mean": rain_mean,
            "std": rain_std,
            "min": rain_min,
            "max": rain_max,
        },
        name="rain (mm/day)",
    )

    return model_counts, rain_summary

model_counts_1, rain_summary_1 = eda_dtype_optimized(csv_path)
model_counts_2, rain_summary_2 = eda_chunked(csv_path)

print("\nTop 10 model counts (approach 1):")
display(model_counts_1.head(10))
print("\nRain summary (approach 1):")
display(rain_summary_1)

print("\nTop 10 model counts (approach 2):")
display(model_counts_2.head(10))
print("\nRain summary (approach 2):")
display(rain_summary_2)
```

```
eda_dtype_optimized:  wall=36.1s  cpu=35.3s  mem Δ+332 MB
eda_chunked:  wall=34.7s  cpu=34.4s  mem Δ+221 MB

Top 10 model counts (approach 1):
model
MPI-ESM1-2-HR    5154240
CMCC-CM2-HR4     3541230
CMCC-CM2-SR5     3541230
CMCC-ESM2        3541230
NorESM2-MM       3541230
TaiESM1          3541230
SAM0-UNICON      3541153
FGOALS-f3-L      3219300
GFDL-CM4         3219300
GFDL-ESM4        3219300
Name: count, dtype: int64
Rain summary (approach 1):
```

```
count    5.924854e+07
mean     1.901170e+00
std      5.585735e+00
min     -3.807373e-12
25%      3.838413e-06
50%      6.154947e-02
75%      1.020918e+00
max      4.329395e+02
Name: rain (mm/day), dtype: float64
Top 10 model counts (approach 2):
model
MPI-ESM1-2-HR    5154240
TaiESM1          3541230
NorESM2-MM       3541230
CMCC-CM2-HR4     3541230
CMCC-CM2-SR5     3541230
CMCC-ESM2        3541230
SAM0-UNICON      3541153
FGOALS-f3-L      3219300
GFDL-CM4         3219300
GFDL-ESM4        3219300
dtype: int64
Rain summary (approach 2):
count    6.246784e+07
mean     1.803193e+00
std      5.456114e+00
min     -3.807373e-12
max      4.329395e+02
Name: rain (mm/day), dtype: float64
```

`@measure` outputs (partners used separate functions *dtype casting* and *column select*; my runs use *dtype + usecols* and *chunked read* as in the cells above):

Member	Approach	OS	RAM	Processor	SSD?	wall (s)	cpu (s)
1 (me)	dtype + usecols	macOS	—	MacBook Air	—	36.1	35.3
1 (me)	chunked read	macOS	—	MacBook Air	—	34.7	34.4
2 (Eric)	dtype casting (`eda_dtype_casting`)	Windows 11	16 GB	Snapdragon X1P42100	256 GB storage	65.0	62.9
2 (Eric)	column select (`eda_select_column`)	Windows 11	16 GB	Snapdragon X1P42100	256 GB storage	36.7	36.1
3 (Raymond)	dtype casting (`eda_dtype_casting`)	macOS	—	MacBook Pro	—	20.6	20.3
3 (Raymond)	column select (`eda_column_select`)	macOS	—	MacBook Pro	—	9.9	9.8

Reflection: On my machine, chunked reading used slightly less RSS change (+221 MB) than dtype + usecols (+332 MB), with almost identical wall time (~35 s) and wall/CPU

ratios near 1, so the work looked CPU-bound rather than I/O-bound. Partner timings diverge a lot: Raymond's dtype casting was much faster than Eric's (20.6 s vs 65.0 s) with a smaller memory increase, and his column-select run was far quicker than Eric's (9.9 s vs 36.7 s), which shows how hardware and OS affect the same code paths. RSS deltas vary wildly (including negative values), in practice I would still use my dtype+usecols or chunked pipeline for this EDA, preferring chunked if peak RAM is the limiting factor.

4. Perform a simple EDA in R

Transfer the combined dataset from Python to R, then run a short exploratory summary in R

Transfer method: Parquet file

I chose the Parquet format because it is a compressed, columnar storage format that is efficiently supported in both Python and R through the arrow package. This allows the dataset to be transferred without loading the entire dataframe into memory in a single process. In contrast, using rpy2 (pandas exchange) would require keeping a large dataframe in memory, which is less efficient for big data. Parquet also makes it easier to reuse the dataset across different tools such as RStudio.

In [ ]:
```
%load_ext rpy2.ipython
repo_root_dir = Path.cwd().parent
combined_dir = repo_root_dir / "data" / "combined"
csv_path = combined_dir / "combined_data.csv"
parquet_path = combined_dir / "combined_data.parquet"

selected_cols = ["time", "rain (mm/day)", "model"]
dtype_map = {
    "rain (mm/day)": "float32",
    "model": "category",
}

if not csv_path.exists():
    raise FileNotFoundError("combined_data.csv not found")

if not parquet_path.exists():
    df_r = pd.read_csv(
        csv_path,
        usecols=selected_cols,
        dtype=dtype_map,
        parse_dates=["time"],
    )
    df_r.to_parquet(parquet_path, index=False)
    print(f"Wrote {parquet_path}")
else:
    print(f"Using existing {parquet_path}")
```

```
parquet_path_str = str(parquet_path.resolve())
```

```
The rpy2.ipython extension is already loaded. To reload it, use:
  %reload_ext rpy2.ipython
Wrote /Users/eric/Documents/MDS/Block 6/525_web-cloud-comp/Lab/Rainfall-Obse
rvation/data/combined/combined_data.parquet
```

In [10]:

```
%%R -i parquet_path_str
suppressPackageStartupMessages({
    library(arrow)
    library(dplyr)
})

rain_r <- read_parquet(parquet_path_str)

# Simple EDA: top models by row count + numeric summary of rainfall
rain_r |>
    count(model, sort = TRUE) |>
    head(10)

rain_r |>
    summarise(
        n = n(),
        mean_rain = mean(`rain (mm/day)`, na.rm = TRUE),
        median_rain = median(`rain (mm/day)`, na.rm = TRUE),
        max_rain = max(`rain (mm/day)`, na.rm = TRUE)
    )
```

```
# A tibble: 1 × 4
         n mean_rain median_rain max_rain
     <int>     <dbl>       <dbl>    <dbl>
1 62467843      1.90      0.0615     433.
```

Challenges

One challenge I encountered was the high memory usage when combining large CSV files using pandas. On some machines, this resulted in significant memory spikes or long execution times. To address this, I explored memory-efficient approaches such as selecting only necessary columns and using chunked reading. These techniques helped reduce memory usage and made the workflow more manageable on a local machine.